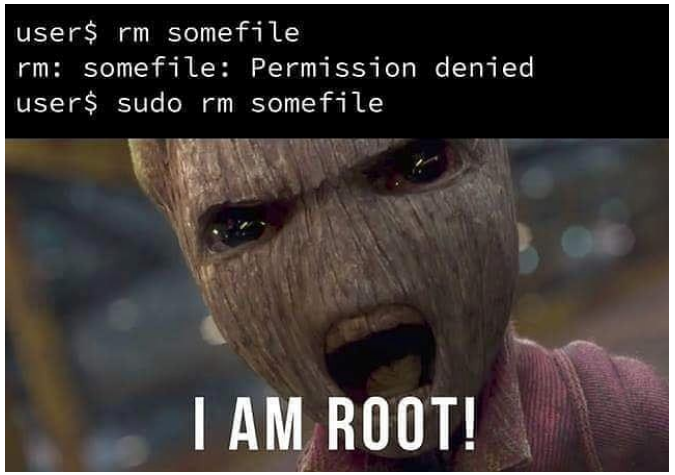
ICT2216/ICT2516C
Secure Software Development



Secure Software Requirements

Raymond Chan & Tram Truong Huu

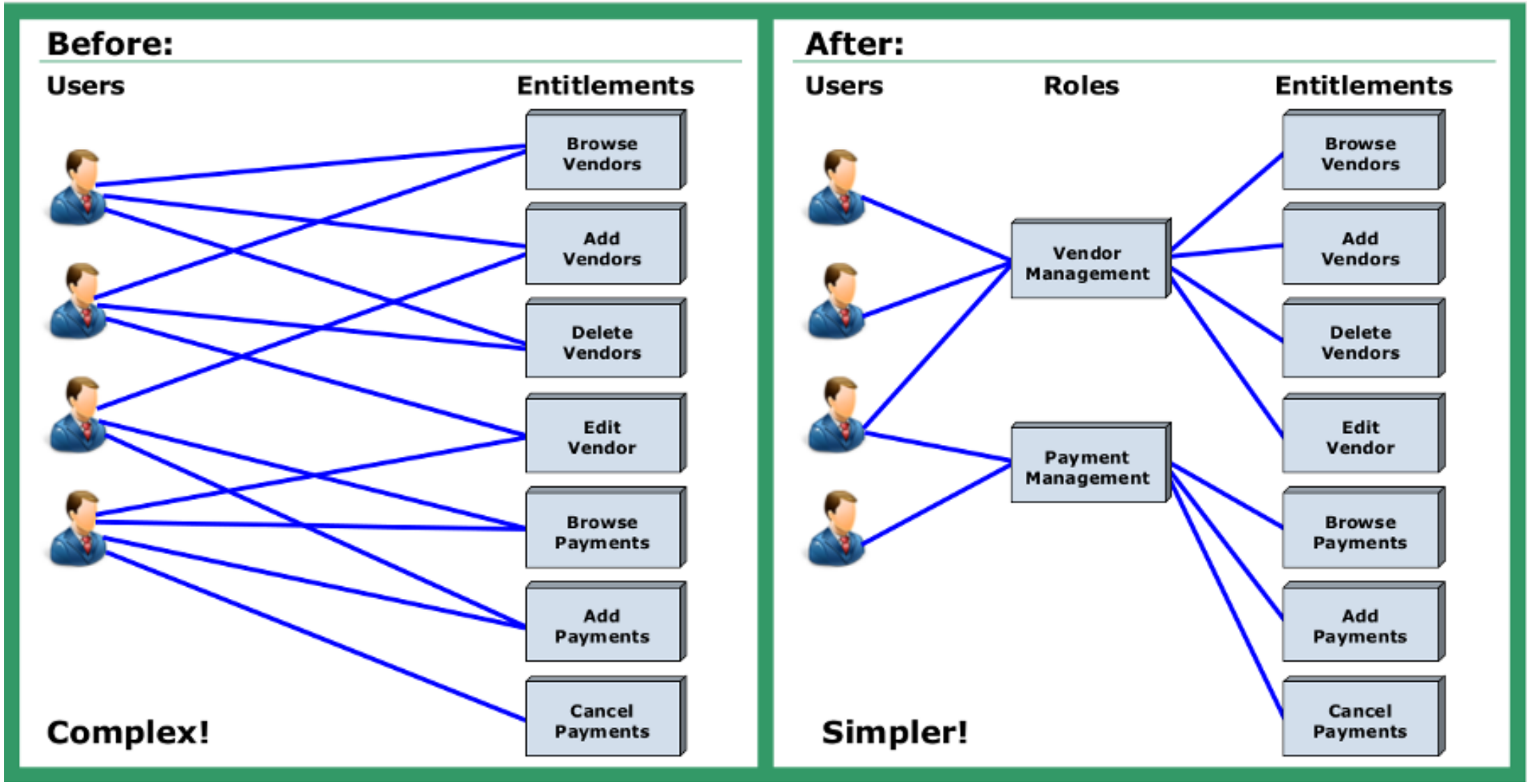
Agenda

- Functional Security Requirements
  - CIA
  - Authentication requirements
  - Authorization requirements
  - Accountability requirements
  - Other security requirement
- Artifact development
  - Abuse/mis-use cases

Authorization requirement

- How are you going to manage the existing users' permissions?
- Defines when a principal may perform an action
  - e.g., Alice is allowed to access her own account but not Bob's account
- There is a wide variety of policies that define what actions might be authorized, e.g., access control policies
  - User-based
  - Role-based

User-based vs. Role-based access control



Examples of Authorization requirements

- *Access to highly sensitive secret files will be restricted to users.*
- *User should not be required to send their credentials each and every time once they have authenticated themselves successfully.*
- *All unauthenticated users will inherit read-only permissions that are part of the guest user role.*
- *Authenticated users will default to having read and write permissions as part of the general user role.*
- *Only members of the administrator role will have all rights as a general user, in addition to having permissions to execute operations.*

Friendly neighborhood spider-man authorized as Avenger

Accountability requirement

- Accountability is all about building records of user actions and act as Detective Control. It helps in detecting when an unauthorized user makes a Change, or when an authorized user makes an unauthorized change.

- Security Requirement specs should clearly define logging and auditing Requirements, How-What-When to capture in accordance with Industry Security Standards and Best Practices.

- It should also define need of Storage, Rotation and Disposal of same.

Accountability requirement

- Retain enough information to be able to determine the circumstances of a breach or misbehavior
- Often stored in log files
  - **Protected from tampering**
  - **Access control**
- Examples
  - "**every action** that you make on the banking website is logged and kept by the bank"
  - "**All failed logon attempts will be logged along with the timestamp** and the Internet Protocol address where the request originated."

General (Application) Security Requirements

From an Application/Software Security perspective, General security requirements should capture proper Session, Error, and Configuration management needs:

- **Session management**
- **Error management**
- **Configuration management**
- **Code management**

Session Management

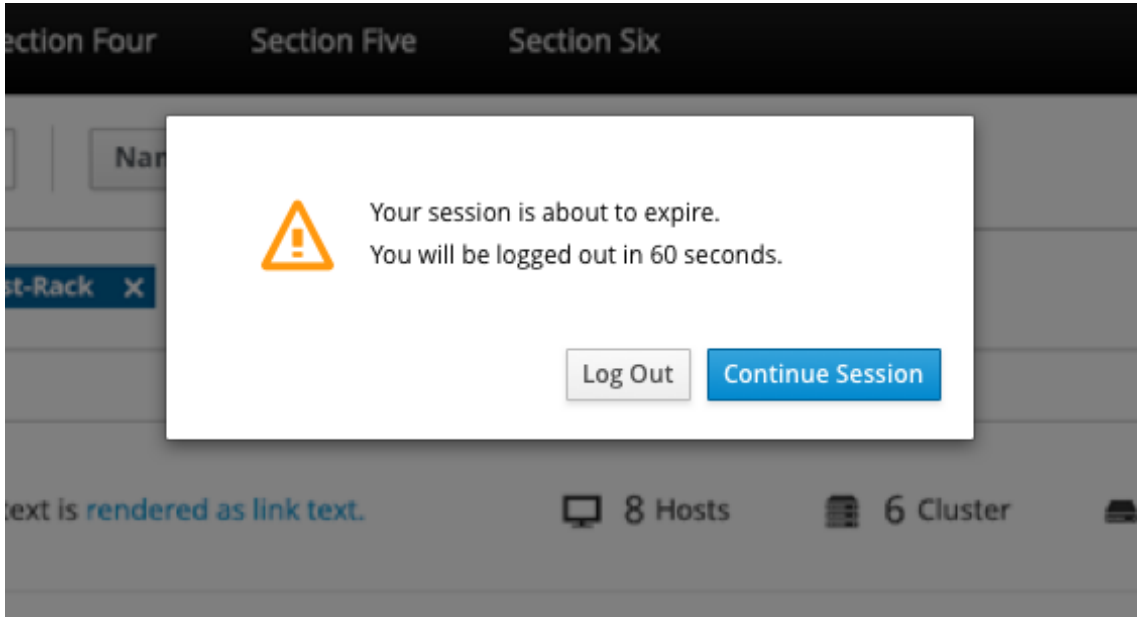
- Sessions are used to maintain state. In usual application communication, on successful user/process Authentication, **Session Identified (ID)** is issued to track the authenticated state.
  - It is compulsory when dealing with Stateless Protocols like **HTTP**.
- Product Security requirement sheet should capture methods and measures required to **Secure Sessions**.
- Requirements defining **Uniqueness and Randomness of Session (Non-Guessable), Expiry, Non-reusability** should be defined.

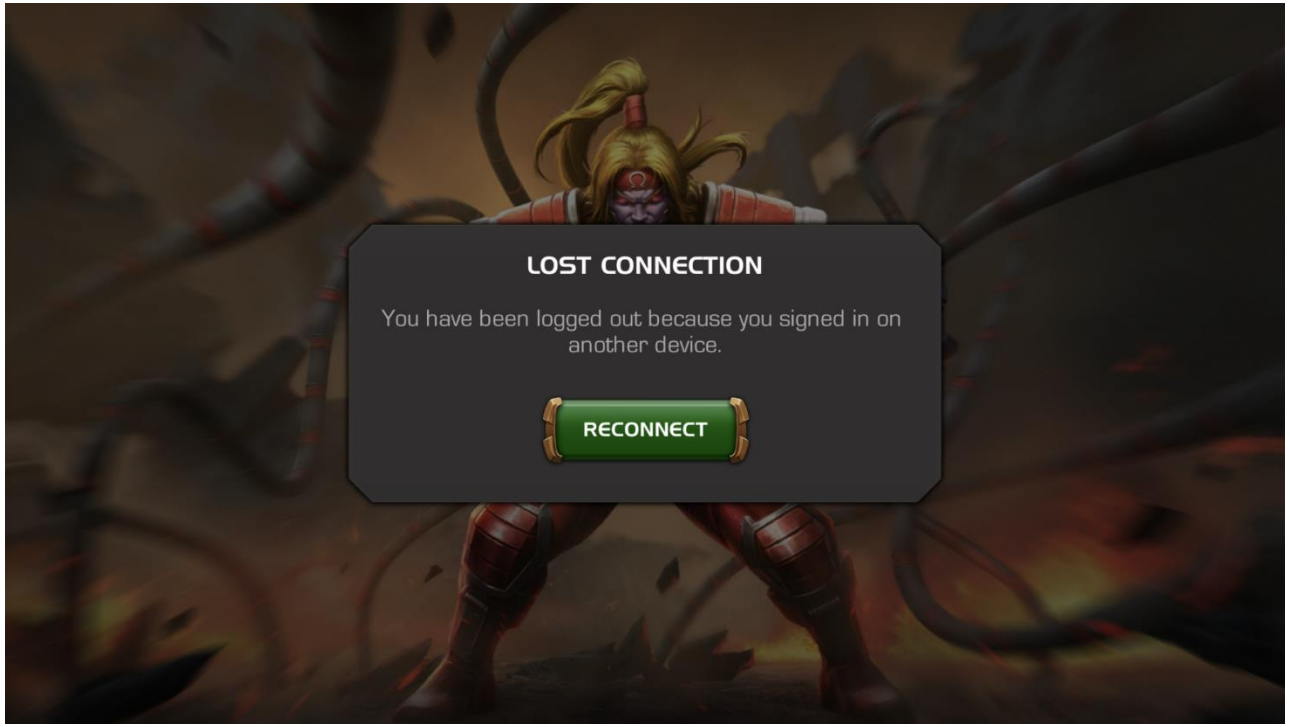
Session Management

- Does your application allow concurrent login?
  - If not, how are you going to handle?
  - If yes, how are you going to handle it?
- Session timeout?
  - How do users feel?

Session Management

- Session Timeout





Error Management

- In Application, providing **Errors and Traces is** part of the usual process when any **unwanted or un-scoped** condition is encountered.
- From Security Prospect, we should clearly define what the level of information should be **to be disclosed/displaced** in such scenarios to avoid **Disclosure Threats** like revealing of any Internal Application architecture, Design and Configuration Information.

Error Management

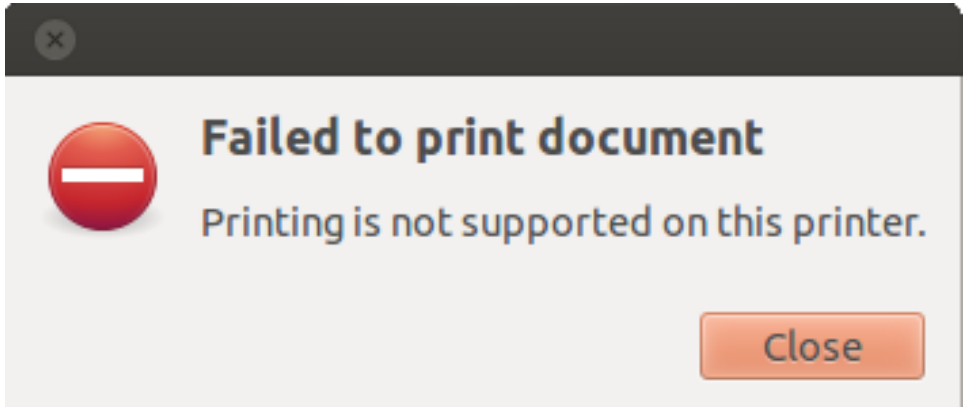
500 Internal Server Error

Sorry, something went wrong.

A team of highly trained monkeys has been dispatched to deal with this situation.

If you see them, show them this information:


```
xYkfidD8Pt6sOtmjpK6_RuJ0Iexo1kl8-kGIpU6inn_ZF4xtAkD2zX_1vUIu
n7eisL-EFKSMVptJn8y2KI6HZOC3e_qPPNrecf7HgMXK4KawZj7wQ-rrJrie
WbeXYsakS0lJ7IuL4OYnTDgaxLbzFaAU_5i5LN5EMjXtGsPeeLM8Se7BzAmQ
xbkmOhq65hVOMCcM0rRQOBuD0e8l1UbyLFBuBVSGLS8LTPLdgb52vDA0JdWD
eNfeY6JRxNxWudtZ4_fKdj2xu87_KI9T3_HsNa0yFOOAXgxlvt3spOKJWTPj
jrIJEoHflgdN-v8mhP5mva6pliOgi44TpRVv7tMVVDlb7DYvkmxlq36W2tbe
aB-GXIRChUlcIm3gqZq7_kxWTFUQuBKeQd8r36cgc2K5NMcFCZScj6UfQKtj
ho8CeQaiAk4-vIHMgHg8IncOVxlAFKY6xpKwaSd_d3DzQca6vKAq6U32C9ja
DretFI4_AiyPq_MbNE13z92kgrlMIiQjB6X_g-GZO2iqV23YwUihnMV5xtbV
lyK6QZe6bXlNHIAPGejBlWkae_kGtaXmoHtuhUPb4NfCq9YXLUx1jlp3wp6J
q4jkLbReZEV7x6F2YRCLam-VoMHE2XLBWQy4Yhl4bDtOldjwNiDwKXdt0XTQ
VCGt5-T49nhSLM4FpGrouOh4O8Zg55wC1XFJnVFTXBOtN3ICY6GPkyrLV19X
efoe4hMz9GxBAti2I4Soz4Q3bFQCFLT_Al05cee3F69ucTMdkkF1lY7bDpMl
eEelf5EOi_InvTcPEQVpOIoGjgf1-KWlXge-h4JQV_fnt5aij2C13VgnqybX
CsQkOK48MbvYwaahO_oouSIhkjItjLDGJWIfpVKKRxK5SEKm7pNtxALXZdWd
```

Configuration management

- Configurations drive application features and functionality. Specific practices and measures should be defined to avoid any **Sensitive Data leakage and Security of these**.

- Measures like **Initialization and Disposal of Global Variables, Hashing/Encryption of Sensitive data** should be clearly defined as a part of configuration.

Code management

- How to store the source code of the project? GitHub?
- How to deploy the project?
- How to prevent someone put **HARDCODE credential** in the source code and uploaded it on GitHub

Silly gits upload private crypto keys to public GitHub projects

Amazing what you can find searching for 'BEGIN RSA PRIVATE KEY'

By John Leyden 25 Jan 2013 at 12:47 38 SHARE ▼

Scores of programmers uploaded their private cryptographic keys to public source-code repositories on GitHub, exposing their login credentials to world+dog. The discovery was made just before the website hit the kill switch on its search engine or, more likely, the service collapsed under the weight of curious users trawling for the sensitive data.

The ability to search for private Secure Shell (SSH) keys on the popular open-source code haven came to light yesterday in tweets and other messages on social networks. At least some of the credentials can still be found using Google and other external web crawlers.

GitHub has more than 2 million users but only a minuscule proportion made the daft mistake of uploading their private instead of just their public crypto keys. Private keys reportedly exposed included the SSH login for a major website in China.

The snafu could allow anyone to surreptitiously log into affected developers' GitHub accounts to alter their projects and gain access to any other online services that use their leaked keys.

Operational Security Requirements

Once the Application/Software is developed and deployed, security should also be considered when it is operational in the environment to avoid any unwanted disclosure or leakage.

- **Deployment Environment**
- **Archiving**
- **Anti-Piracy**

Deployment Environment

- The Security Requirement list should capture information about the environment in which the software will be deployed and who will be using it.
- Environment Compliance and Industry Standard requirements are driven by this information.
- Not only the production environment, but the development environment…
- Prof Raymond Chan will talk about the details in 2nd half of the lecture ☺!

Archiving

- Archiving is required to ensure Business Continuity, Regulatory Requirements, and Organizational (Retention) Policy.

- It is important to capture archiving requirements to comply with the organization's policy and regulations.

- Measures like Where (media type) and How (online/offline, format, encryption) Data will be stored, Data retrieval policy, etc., should be clearly defined as part of the requirement sheet.

Anti-Piracy

- It is a part of the Commercial off-the-shelf (COTS) requirement. It includes Code Obfuscation, Signing, Anti-Tampering, Licensing, and IP Protection mechanisms.

- This requirement should be clearly addressed during Requirement Gathering to avoid any future discrepancies and legal obligations.

Other Security Requirements…

Sequencing & Timing:

- Sequencing and Timing design flaws can lead to Race Conditions or TOC/TOU Attacks. Race Windows, usage of Atomic Operations or Mutex Requirements (Mutual Exclusions or Resource Locking) must be identified as a part of the Security Requirement

Procurement Needs:

- This is required to get proposals from qualified contractors. It should specify the scope of the desired procurement, define the evaluation process, and delineate the deliverables and requirements associated with the project.
- Selection of IDE, libraries, and programming language

International Regulation:

- International Regulation and Compliance needs should also be discussed and captured as a part of Security Requirement Gathering, e.g., GDPR.

DEVELOP ARTIFACTS

Develop artifacts

General principle: Once you know what a system does, look at it from the adversary's perspective

- How can they disrupt the system?
- How can they profile from the system?

Two main techniques:

- Anti-requirements
- **Abuse / Misuse case modeling**

22

Anti-requirements

Requirements generally have the form

- The system shall ***[do something]*** given ***[inputs]***

Anti-requirements: the things that you don't want your system to do.

- To develop an **anti-requirement**
  - Consider undesirable outcomes that are possible –what outcomes are **unacceptable**
  - Explore the inputs and determine the outcome associated with

An example of “anti-requirements”

*Requirement: The system **shall** produce a unique identifier valid for N days into the future **given** a time, an integer N: 0 < N ≤ 7, and a valid authorization token*

Consider Undesirable Outcomes

- System crash
- Non-unique identified produced
- Identifier with an incorrect validity period
- …

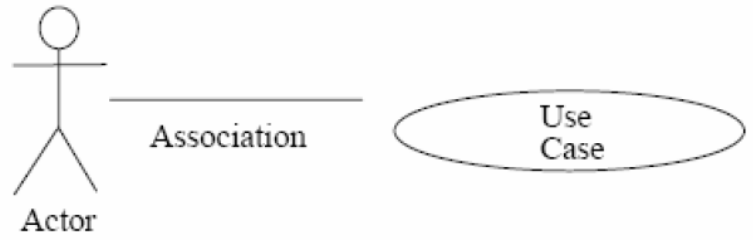
Consider Inputs

- Time mis-formatted
- N ≤ 0 or N > 7
- Invalid authorization token

Abuse / Misuse Case Modeling

Abuse cases can be used to illustrate the security requirements

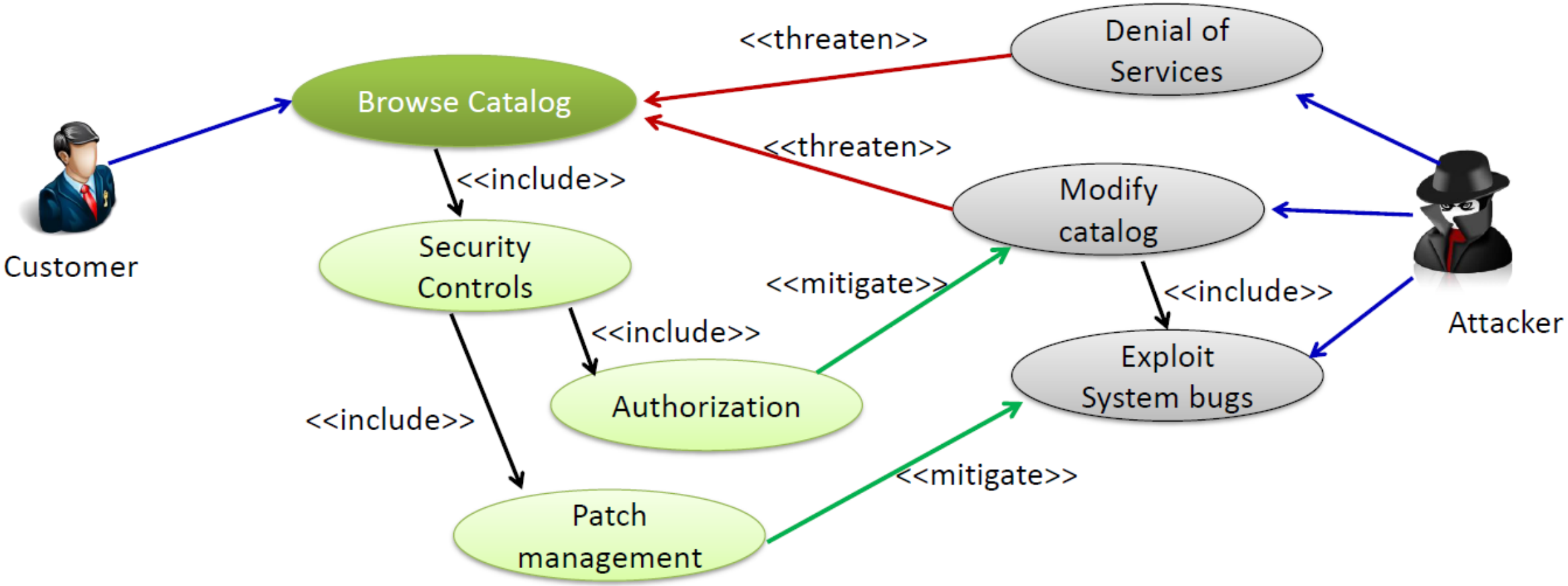
- use cases describe what a system **should do**
- abuse cases describe what it **should not do**



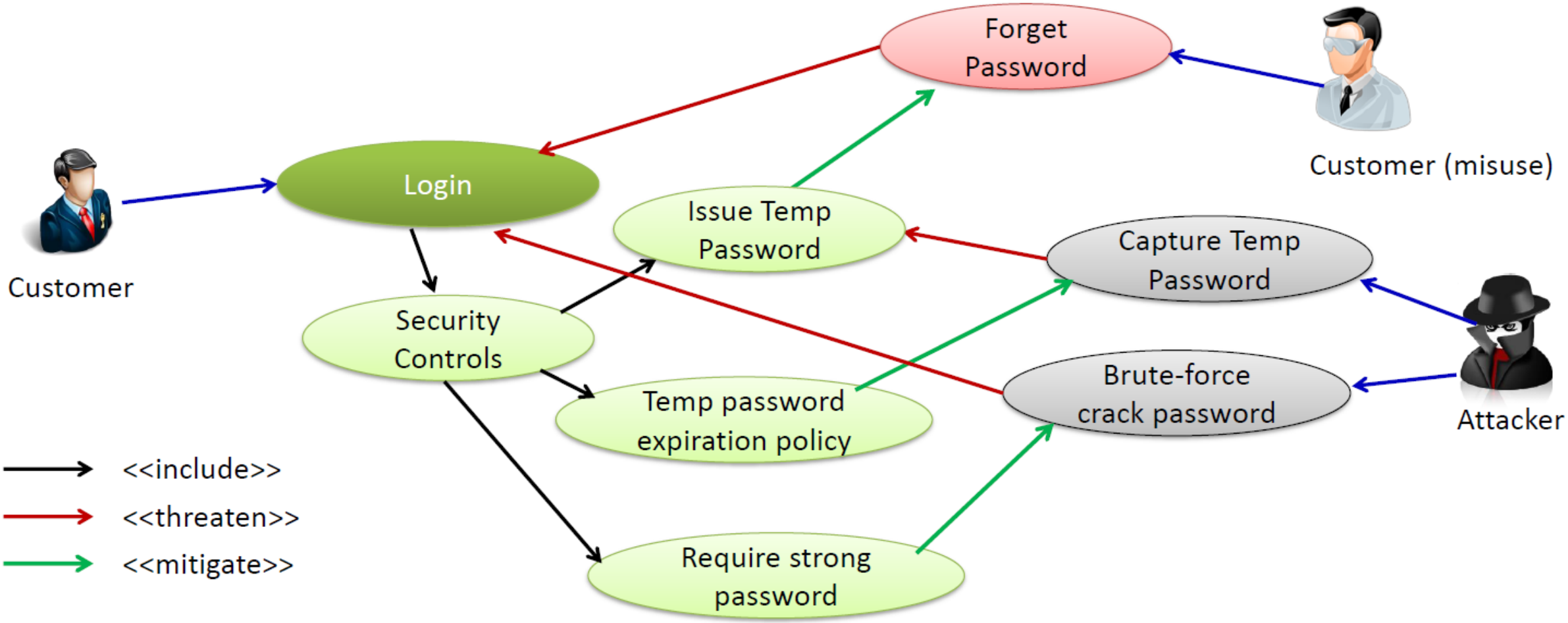
How? Start with use cases

- Consider how the process can be interdicted or corrupted
  - Identify attackers (like identifying actors in use cases)
- Model using simple graphics (UML is useful)
- Update with security requirements

Example of abuse case: Browse Web catalog



Example of abuse / misuse case: login



Exercise case study: A simple ATM

Scenarios

- Login
- Withdraw Money
- Logout

1. Consider a few functional requirements, and draw a **use case diagram**
2. Consider **abuse / misuse cases and extend the use case diagram with abuse / misuse cases**
   - Shoulder snooping
   - Steal card
   - Copy card

ATM security requirements

What are the security requirements

- Input Validation
- Velocity
- Transactions
- Visibility

Input Validation : Different Levels

Length and Boundaries (PIN number)

- 6 input fields
- 0-9 inclusive

Characters and encoding

- English characters in ASCII or Unicode
  - 4 languages used in Singapore

Syntactic

- Only positive numbers are allowed for withdrawal

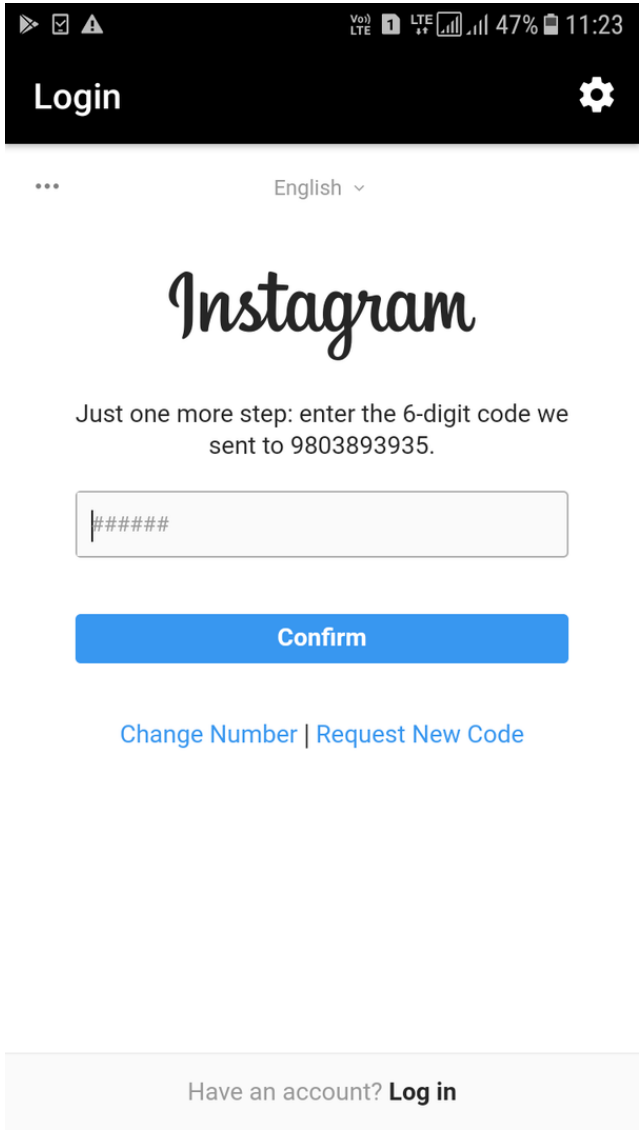
Semantic

- Total withdrawal should not be greater than the account balance

Velocity Checking

How many shots does an attacker get?

- At what rate?
  - Number of logins/hour
    - Gmail blocks after repeated attempts
  - Transactions/minute
  - Kilobytes/day
  - Changes/user
- PIN blocked after 3 wrong attempts



Transactions

- Operations can be interrupted
- Just because you start, it doesn’t mean you finish
- What happens if the user forgot to take out the ATM card?
  - Alerts user with sounds
  - Do not dispense cash unless card is removed
  - The machine swallows it to prevent misuse

Visibility

- Do not display the PIN as it is typed
- Do not display account information once the card is removed

SINGAPORE INSTITUTE OF TECHNOLOGY

SOFTWARE SECURITY FRAMEWORK

Introduction to Software Security Framework

- How can I ensure the application is secure in the design phase?
- Follow international standards when you are designing your application!

- Some popular frameworks:
  - Payment Card Industry Software Security Framework
  - NIST Software Development Framework

Payment Card Industry Software Security Framework

- One of the hardest security requirements. (Why?)

- The security concepts described within this document collectively protect payment transactions and data, minimize vulnerabilities, and defend payment software from attacks throughout the software lifecycle.

PCI SSF Requirement Example

Requirement:

- **Sensitive production data is protected** when retained on vendor systems and securely deleted when no longer needed.

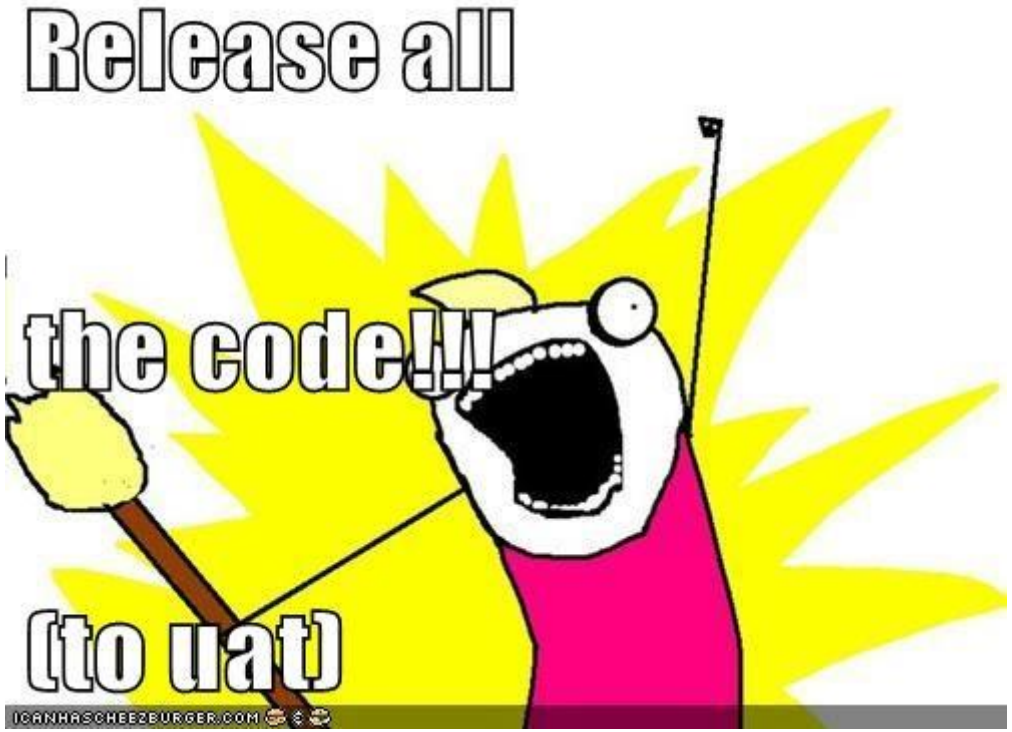
Test Requirements:

- The assessor shall **examine vendor evidence and interview personnel to confirm that a mature process exists to ensure sensitive production data is protected** when retained on vendor systems and is securely deleted when no longer needed.
- The assessor shall examine vendor evidence and observe a sample of vendor systems to confirm the following:
  - Sensitive production data is not resident on vendor systems unless appropriate evidence of approval and justification exists.
  - Sensitive production data is appropriately protected where it is retained.
  - Secure deletion processes or mechanisms are sufficient to render sensitive production data irretrievable.

Question time

- Can I use **real customer data** in **development environment**?

- UAT = USER AS A TESTER?



NIST Secure Software Development Framework

- Intended to facilitate communications about **secure software development practices** amongst business owners, software developers, and cybersecurity professionals within an organization

- Provide a core set of **high-level secure software development** practices

- https://nvlpubs.nist.gov/nistpubs/CSWP/NIST.CSWP.04232020.pdf

NIST SDF Requirement Example

- **Design Software to Meet Security Requirements and Mitigate Security Risks (PW.1)**:
  - **Identify and evaluate the applicable security requirements** for the software's design; determine what security risks the software is likely to face during production operation, and how those risks should be mitigated by the software's design.
  - **Justify** any cases where risk-based decisions conclude that security requirements should be relaxed or waived. **Addressing security requirements and risks during software design (secure by design)** helps to make software development more efficient.

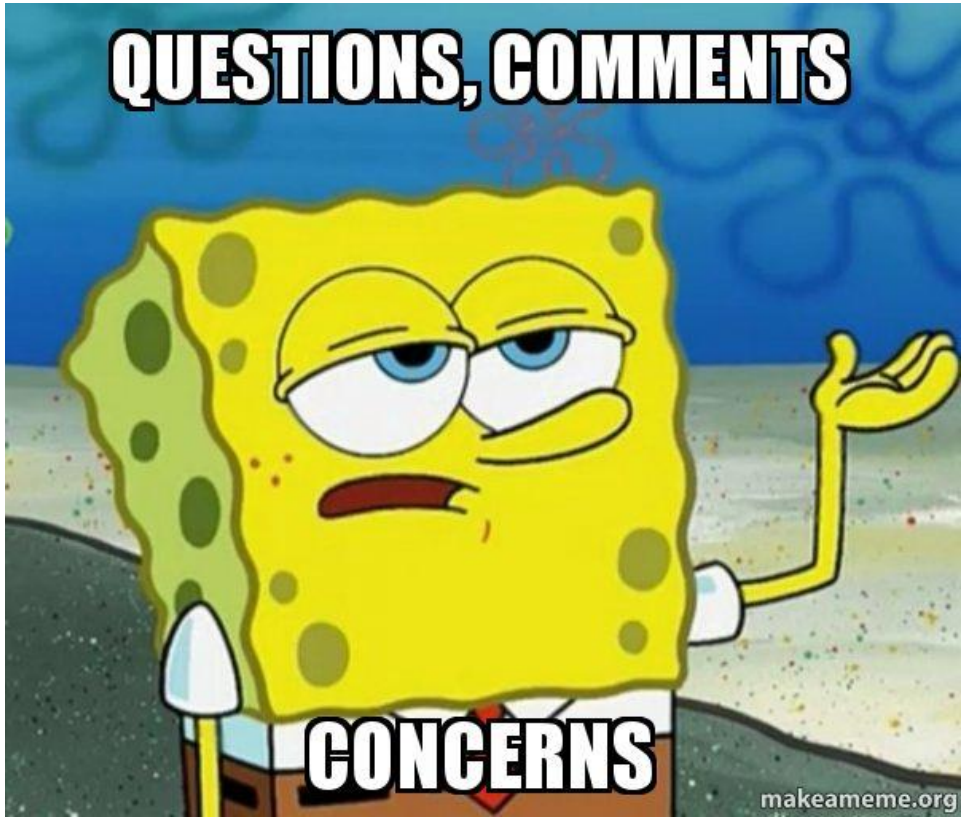
NIST SDF Requirement Example

- **Train the development team** (the security champions in particular) or collaborate with a threat modeling expert to **create threat models** and **attack models** and to analyze how to use a risk-based approach to address the risks and implement mitigations.
- Perform more rigorous assessments for high-risk areas, such as **protecting sensitive data** and **safeguarding identification, authentication, and access control, including credential management**.
- Review **vulnerability reports** and statistics for previous software.

Conclusion

- To conclude:
  - Introduction to Secure Software Requirements
    - Confidentiality
    - Integrity
    - Availability
    - Authentication
    - Authorization
    - Accountability

- Before you design the application, PLEASE consider security requirements!!



QUESTION AND COMMENTS?

Reference

- https://www.pcisecuritystandards.org/documents/PCI-Secure-SLC-Standard-v1_0.pdf
- https://www.ansi.org/news_publications/news_story?menuid=7&articleid=8fe6ed5f-6ae8-4e92-9f2e-c72c27c4be85
- https://software-security.sans.org/resources/swat
- https://www.bankinfosecurity.com/whitepapers/best-practices-for-implementing-nist-password-guidelines-special-w-5593